

FlyWise (APIx) — Antigravity Prompt Sequence

How to use this document:

1. Keep **one continuous Antigravity chat session** for the entire build — don't start a new session per prompt. Later prompts assume earlier files already exist in that session's context.
 2. Paste the **Shared Project Context** block below as your very first message to Antigravity, before Prompt 1. This grounds the agent in every locked decision so it doesn't invent or drift from anything we designed.
 3. Then send each numbered prompt **one at a time, in order**. Wait for Antigravity to finish and show you the result before sending the next one.
 4. After each prompt, a  **Verify** line tells you exactly what to check before moving on. Don't skip this — catching a mistake at prompt 3 is much cheaper than finding it at prompt 15.
 5. **MANUAL STEP** blocks are things *you* do in a browser (Render/Vercel/Neon dashboards) — these are not Antigravity prompts, don't paste them into the agent.
-

SHARED PROJECT CONTEXT — paste this once, first

You are helping me build "FlyWise" (technical index name: APIx), a real-time
airfare
price index prototype for Smart India Hackathon 2026, problem statement
PS26056
(MoSPI/DIID – Real-time Airfare Price Index for India).

LOCKED ARCHITECTURE – do not deviate from this without asking me:

- Stack: Python (collectors + ingestion + processing), FastAPI (backend),
React/Next.js + TailwindCSS + Framer Motion (frontend), PostgreSQL (Neon,
cloud-hosted)
- Hosting: Render (2 Cron Jobs + 1 Web Service), Vercel (frontend), Neon (DB)
- - everything cloud-hosted, nothing depends on a local machine staying on
 - NO Pentaho, NO ETL GUI tool – ingestion/normalization/dedup is plain Python
 - NO synthetic/fabricated data, ever, under any circumstance. Every
observation must
come from a real API call (Bright Data, Scrappa, or direct OTA scrape). If
a data
source fails, that's a missing observation to be imputed (see imputation
rules below)
 - never invented from scratch.
- Data sources: Bright Data (Google Flights Scraper API, primary), Scrappa
(Google
Flights JSON, secondary), 1-2 direct OTA Playwright scrapes (source-
consensus check)
- Collection frequency: every 4 hours (6x/day), for BOTH domestic and
international
routes – synced, not split.

ROUTE BASKET (12 core + 2 seasonal, all confirmed direct/nonstop, no
connecting flights):

Domestic: DEL-BOM, DEL-BLR, BOM-BLR, DEL-CCU, BLR-HYD, MAA-DEL, BOM-HYD, DEL-
AMD

International (direct only): DEL-DXB (AED), BOM-DXB (AED), DEL-SIN (SGD),
BOM-SIN (SGD)

Seasonal (separate analytics tranche): DEL-GOI (winter leisure, Oct-Mar),
DEL-SXR (summer leisure, Apr-Oct)

LEAD-TIME WINDOWS:

- Domestic: T+1, T+7, T+15, T+21 (CPI-compatible), T+30, T+45 (analytical)
- International: T+1, T+30 (analytical), T+60 (CPI-compatible)

CURRENCIES: INR is primary (all domestic + the index itself). AED and SGD for

the international routes, FX-locked at collection time using RBI/FBIL daily reference rates (never re-converted later). USD is display-layer only – it NEVER enters the index calculation, it's purely a "show this in dollars" toggle for the dashboard.

DATABASE SCHEMA (already finalized – use exactly this):

```
CREATE TABLE routes (  
    route_id VARCHAR(10) PRIMARY KEY,  
    origin_airport VARCHAR(3) NOT NULL,  
    destination_airport VARCHAR(3) NOT NULL,  
    domestic_international VARCHAR(15) NOT NULL,  
    route_weight NUMERIC,  
    currency VARCHAR(3) DEFAULT 'INR',  
    is_seasonal BOOLEAN DEFAULT FALSE,  
    season_window VARCHAR(30)  
);  
  
CREATE TABLE observations (  
    observation_id UUID PRIMARY KEY,  
    route_id VARCHAR(10) REFERENCES routes(route_id),  
    service_type VARCHAR(10) NOT NULL,  
    collection_timestamp TIMESTAMP NOT NULL,  
    travel_date DATE NOT NULL,  
    advance_days INT NOT NULL,  
    window_category VARCHAR(20) NOT NULL,  
    airline VARCHAR(10),  
    flight_number VARCHAR(10),  
    operating_carrier VARCHAR(10),  
    cabin VARCHAR(20) DEFAULT 'economy',  
    fare_family_raw VARCHAR(50),  
    fare_family_tier VARCHAR(20),  
    baggage_bucket VARCHAR(10),  
    service_spec_id VARCHAR(100),  
    base_fare NUMERIC,  
    taxes NUMERIC,  
    mandatory_fees NUMERIC,  
    total_fare_original_currency NUMERIC,  
    original_currency VARCHAR(3),  
    fx_rate_used NUMERIC,  
    fx_rate_date DATE,  
    total_fare_inr NUMERIC,
```

```

availability_status VARCHAR(20),
is_imputed BOOLEAN DEFAULT FALSE,
imputation_method VARCHAR(30),
quality_score NUMERIC,
source VARCHAR(30),
collection_method VARCHAR(30),
run_id VARCHAR(50),
raw_reference TEXT,
methodology_version VARCHAR(10) DEFAULT 'v1.0'
);

CREATE TABLE flight_events (
    event_id UUID PRIMARY KEY,
    flight_number VARCHAR(10),
    travel_date DATE,
    operational_status VARCHAR(20),
    logged_at TIMESTAMP
);

CREATE TABLE route_index (
    route_id VARCHAR(10) REFERENCES routes(route_id),
    window_category VARCHAR(20),
    advance_days INT,
    date DATE,
    price_relative NUMERIC,
    n_observations INT,
    PRIMARY KEY (route_id, window_category, advance_days, date)
);

CREATE TABLE national_index (
    date DATE,
    window_category VARCHAR(20),
    domestic_apix NUMERIC,
    international_apix NUMERIC,
    overall_apix NUMERIC,
    status VARCHAR(15),
    coverage_score NUMERIC,
    confidence_score NUMERIC,
    PRIMARY KEY (date, window_category)
);

INDEX FORMULA:
- Route-level: Jevons geometric mean of price relatives –

$$I_{t} = ( \text{product of } (P_{i,t} / P_{i,t-1}) \text{ for } i=1..n )^{(1/n)} * 100$$

- Aggregation: Young-type fixed-weight formula –

```

```

    APIx = sum( route_weight_r * route_index_r )
- Compute DAPIx (domestic) and IAPIx (international) as SEPARATE series
first,
    then combine into overall APIx – never blend domestic and international
    observations into one Jevons calculation.
- Base period: Day 1's collection runs (median per route/window) – NOT a 5-7
day
    window, since this is a compressed 5-6 day build.
- Constant-Quality Pricing: only ever compare price relatives within the same
    service_spec_id (FSID). FSID = route_id + "-" + cabin + "-" + service_type
+
    "-T" + advance_days + "-" + baggage_bucket + "-" + fare_family_tier
    Fare family tiers are hardcoded per-airline lookups: "base" | "standard" |
    "flex"
    – no hedonic model, no general classifier.

```

EDGE CASES TO HANDLE (all locked, don't skip any):

1. Diverted/incomplete flights: logged in `flight_events.operational_status`, NEVER affects observations or the price index.
2. Partner-airline substitution after cancellation: treated as ordinary matched-replacement (same route/cabin/directness/lead-time/baggage), priced at the replacement flight's real market fare – never modeled as a "free" rebooking discount.
3. Missing data imputation, in this exact priority order, each result tagged with `is_imputed=true` and the imputation_method used:
 - a) `neighbor_window` – same route, same day, infer from other available windows
 - b) `time_carry_forward` – same route+window, most recent successful run
 - c) `cross_route` – same domestic/international bucket, other routes' average movement
 - d) `full_carry_forward` – last resort, reuse last known absolute price
4. Session-contamination control: every scrape uses a clean/incognito session, never reused cookies.
5. Codeshare dedup: dedupe by `operating_carrier` + `travel_date` + approximate departure time, NOT by marketing flight number.
6. Glitch-fare flagging: flag (don't auto-delete) any `total_fare_inr` below 20% of that route's trailing 7-day median for review; require `base_fare + taxes + mandatory_fees ≈ total_fare_original_currency` (reconciliation check) before accepting an observation.
7. Flight renumbering: match by `route_id` + `travel_date` + approximate departure time, treat `flight_number` as secondary/display only.

```
8. Seasonal routes: is_seasonal=true routes with no available flights outside
their season_window get availability_status='route_unavailable', never
'source_error' or 'sold_out'.
```

Confirm you understand this context before I send the first build prompt.

✅ **Verify:** Antigravity should respond confirming it understood the architecture, schema, and edge cases — not asking you to re-explain something already covered above. If it seems to have missed something (e.g., asks "what database are we using"), re-paste the relevant part before continuing.

MANUAL STEP — Day 1: Accounts (do this yourself, not a prompt)

Before any code, get these done (see the Master Build Plan doc, §2, for full detail):

1. GitHub — new private repo (`apix-sih-ps26056`)
2. Neon — new project, copy the connection string
3. Bright Data — sign up, create a Google Flights Scraper API zone, get the API token
4. Scrappa — sign up, get the API key
5. Render — sign up, connect GitHub (don't create services yet)
6. Vercel — sign up, connect GitHub (don't deploy yet)

✅ **Verify:** you have the Neon connection string, Bright Data token, and Scrappa key saved somewhere safe (not in any file you'll commit to git).

PROMPT 1 — Repo scaffold + DB schema

Set up the initial repo structure for this project. Create these folders with placeholder/empty files where noted:

```
apix-sih-ps26056/
├─ collectors/
│   └─ raw_output/{domestic,international}/  (empty folders, .gitkeep
files)
├─ ingestion/
│   └─ lookup_tables/                        (empty folder, .gitkeep)
├─ processing/
├─ api/
│   └─ routers/
│   └─ middleware/
├─ dashboard/
├─ db/
│   └─ migrations/
├─ docs/
├─ config/
└─ README.md
```

In db/schema.sql, write the exact CREATE TABLE statements from the schema in my shared context above – all 5 tables (routes, observations, flight_events, route_index, national_index), verbatim, no changes.

In config/.env.example, create a template listing every environment variable we'll need (with placeholder values, not real secrets):
NEON_DATABASE_URL, BRIGHT_DATA_API_TOKEN, BRIGHT_DATA_ZONE, SCRAPPA_API_KEY, API_KEY_ALLOWED_LIST (for our own policy API auth later).

In config/.gitignore, add standard Python + Node ignores, plus explicitly:
.env,
collectors/raw_output/*.json, __pycache__/, node_modules/, .next/.

Write a short README.md explaining what this project is (2-3 sentences: FlyWise, a real-time airfare price index for India, SIH 2026 PS26056 prototype) and how the repo is organized (one line per top-level folder).

Do not write any business logic yet – this prompt is scaffolding only.

✅ **Verify:** open the repo, confirm all folders exist, `schema.sql` matches the shared context exactly (compare table-by-table), `.env` is in `.gitignore` (critical — don't skip this check).

MANUAL STEP: run `db/schema.sql` against your Neon database now (via Neon's SQL editor in their web console, or `psql` if you have it). Confirm all 5 tables were created by listing tables in Neon's console.

Also create `db/seed_routes.sql` — ask Antigravity for this as part of the same prompt if it didn't include it, with one INSERT per route from the route basket (12 core + 2 seasonal) in the shared context, filling in

`origin_airport`/`destination_airport`/`domestic_international`/`currency`/`is_seasonal`/`season_window` correctly for each. Run this against Neon too once `schema.sql` succeeds.

PROMPT 2 — Bright Data collector

```
Write collectors/bright_data_adapter.py. This module:
```

1. Reads BRIGHT_DATA_API_TOKEN and BRIGHT_DATA_ZONE from environment variables
(use python-dotenv for local testing).
2. Exposes a function `fetch\_fares(origin: str, destination: str, travel\_date: str, advance\_days: int) -> dict` that calls Bright Data's Google Flights Scraper API
(POST to <https://api.brightdata.com/request>, with the zone, target URL, and format:"json" in the payload, Authorization header with the Bearer token).
3. Parses the raw response into a flat dict matching these observation fields:
airline, flight_number, operating_carrier, cabin, fare_family_raw, base_fare, taxes, mandatory_fees, total_fare_original_currency, original_currency, availability_status, stops (to confirm it's a direct/nonstop flight – discard or flag any result that isn't direct, since our whole route basket is direct-only).
4. Writes each parsed result as a JSON file into
`collectors/raw_output/{domestic|international}/{route_id}_{advance_days}_{times}`
– pick domestic vs international based on whether the route is in our international list (DEL-DXB, BOM-DXB, DEL-SIN, BOM-SIN).
5. Includes basic error handling: on API failure or timeout, log the failure (route, window, timestamp, error) to a local errors.log instead of crashing –
this failure is what our imputation system will later detect and fill in.
Do NOT generate any fallback/placeholder price data on failure – just log and skip.
6. Add a `if \_\_name\_\_ == "\_\_main\_\_":` block that runs one test fetch for DEL-BOM, T+21, and prints the result, for manual testing before we wire up the full scheduler.

Use the `requests` library. Add type hints throughout.

✅ **Verify:** run the test block locally (`python collectors/bright_data_adapter.py`) with your real Bright Data token in a local `.env`. Confirm it returns real DEL-BOM fare data, not an error, and that a JSON file actually appears in `raw_output/domestic/`.

PROMPT 3 — Scrappa collector

```
Write collectors/scrappa_adapter.py, following the exact same structure and function signature as collectors/bright_data_adapter.py (fetch_fares(origin, destination, travel_date, advance_days) -> dict), but calling Scrappa's Google Flights JSON API instead of Bright Data's. Use SCRAPPA_API_KEY from environment variables. Same output format, same file-writing convention, same error-logging behavior (log and skip on failure, never fabricate data), same direct-flight-only filtering. Set source field to "scrappa" instead of "bright_data" in the output.
```

✅ **Verify:** same as Prompt 2 — run its test block, confirm a real result and a JSON file written.

PROMPT 4 — OTA direct-scrape collector

```
Write collectors/ota_scraper.py using Playwright (Python). This scrapes
Ixigo's
search-results pages directly (try Ixigo first; if it blocks scraping or
shows
persistent CAPTCHA challenges, switch to EaseMyTrip instead – do not attempt
to
bypass either), for 1-2 routes only (we'll configure which in
routes_config.json
later). Requirements:

1. Uses Playwright's async API, launches a fresh browser context per search
  (no cookie/session reuse across calls – this is a locked data-integrity
  requirement, not optional).
2. Respects the site's robots.txt – check it programmatically before scraping
  and abort with a clear error if disallowed.
3. No CAPTCHA-solving, no anti-bot-bypass tooling, no IP rotation – if the
  site
  blocks the request, log the failure and skip, exactly like the API
  adapters do.
4. Parses whatever fare/flight fields are visible on the public search-
  results
  page into the same flat dict structure as the other two adapters.
5. Same file-writing convention into collectors/raw_output/.
6. Set source field to "ota_playwright".

Before writing the scraper logic, first fetch the site's actual search-
results
page structure so the parsing selectors are based on the real DOM, not
guessed.
```

✅ **Verify:** this is the riskiest collector to get working (site layouts change, may need real debugging). Run it manually, confirm it returns real data for at least one test route without tripping any bot protection. If the chosen OTA site turns out to be heavily protected, tell me and we'll pick a different one from the Ixigo/EaseMyTrip/Cleartrip options.

PROMPT 5 — Cron entrypoint (orchestrator)

Write `collectors/render_cron_entrypoint.py`. This is the single script
Render's
Cron Job will execute every 4 hours. It should:

1. Load `routes_config.json` (create this file too – structure it with three arrays: "domestic" (8 routes, each with `route_id` and windows: `[1,7,15,21,30,45]`),
"international" (4 routes, each with `route_id`, `currency`, windows: `[1,30,60]`),
and "seasonal" (2 routes, each with `route_id` and `season_window`) – using the exact route list from my shared context).
2. For each domestic and international route (skip seasonal routes unless today's date falls inside their `season_window`), and each of that route's windows, calculate the target `travel_date` (today + `advance_days`) and call `bright_data_adapter.fetch_fares()` as primary. If Bright Data fails/errors, fall back to `scrappa_adapter.fetch_fares()` for that specific route/window. Do not fall back to any fabricated data if both fail – just log and move on.
3. For the 1-2 OTA-scrape routes, also call `ota_scraper.py` once per cycle.
4. Logs a summary at the end: how many observations succeeded, how many failed, broken down by route.
5. This script does NOT write to Postgres directly – that's the ingestion pipeline's job (next prompt). This script only produces the raw JSON files.

Add clear logging throughout (route, window, timestamp, success/failure) so we can debug collection issues from Render's log viewer once deployed.

✅ **Verify:** run this locally once, end to end. Confirm JSON files land in `raw_output/` for all 12 core routes across their respective windows, and that the summary log looks sane (mostly successes, maybe a few failures which is normal).

PROMPT 6 — Ingestion pipeline (replaces Pentaho)

Build the full ingestion pipeline in the ingestion/ folder – four scripts, run in this exact order by a single orchestrator function:

1. ingestion/ingest_raw_json.py – reads every JSON file currently in collectors/raw_output/{domestic,international}/, parses them into a list of dicts matching the observations table columns, and deletes/archives each file after successful parsing (move to raw_output/processed/ rather than deleting, so we keep the raw evidence trail per our auditability requirement).
2. ingestion/normalize_airports.py – create ingestion/lookup_tables/airport_codes.csv with columns: raw_code, normalized_code, city_name – seed it with our 12+2 routes' airports (DEL, BOM, BLR, CCU, HYD, MAA, AMD, DXB, SIN, GOI, SXR) mapped to themselves (they're already clean IATA codes from our sources, but this lookup step exists to catch any inconsistent naming from different sources).
The script does a lookup-and-standardize pass on each parsed observation.
3. ingestion/normalize_fare_family.py – create ingestion/lookup_tables/airline_fare_family_map.csv with columns: airline, fare_family_raw, fare_family_tier (base|standard|flex), baggage_bucket (0kg|15kg|20kg+). Leave this CSV with just a header row and 2-3 placeholder example rows for now (e.g., IndiGo,Saver,base,0kg) – I'll fill in real fare names once we see actual scraped data in testing. The script looks up each observation's fare_family_raw against this table to set fare_family_tier and baggage_bucket, then constructs service_spec_id exactly as: route_id + "-" + cabin + "-" + service_type + "-T" + str(advance_days) + "-" + baggage_bucket + "-" + fare_family_tier. If a fare_family_raw isn't found in the lookup table, tag it with fare_family_tier="unmapped" rather than guessing.
4. ingestion/dedup_observations.py – drops duplicate observations from the current batch, keyed on (route_id, travel_date, operating_carrier,

```
collection_timestamp rounded to the nearest hour) – this specifically catches codeshare duplicates (same physical flight, different marketing flight numbers) since we key on operating_carrier, not flight_number.
```

5. Write an orchestrator function `run_ingestion_pipeline()` that calls all four in order, then inserts the final cleaned batch into the observations table in Postgres (use `psycopg2` or `asyncpg`, connection string from `NEON_DATABASE_URL` env var). Each inserted row's `raw_reference` field should point to the archived raw JSON file path for auditability.

Wire this orchestrator into `collectors/render_cron_entrypoint.py` so it runs right after the collection step, in the same script/cron execution.

✅ **Verify:** run the full pipeline locally end to end (collection → ingestion → Postgres insert). Query your Neon `observations` table directly and confirm real rows appear with sensible values in every column, especially `service_spec_id` (spot-check the format matches exactly).

MANUAL STEP — Deploy Cron Job 1 to Render

1. In Render's dashboard, create a new **Cron Job**, connect it to your GitHub repo.
2. Set schedule to `0 */4 * * *` (every 4 hours).
3. Set the start command to run `collectors/render_cron_entrypoint.py`.
4. Add environment variables: `NEON_DATABASE_URL`, `BRIGHT_DATA_API_TOKEN`, `BRIGHT_DATA_ZONE`, `SCRAPPA_API_KEY`.
5. Trigger a manual run from Render's dashboard to test it before waiting for the schedule.

✅ **Verify:** check Render's logs for that run, and check your Neon `observations` table for new rows with a recent `collection_timestamp`.

PROMPT 7 — Quality/FX engine

Write `processing/quality_fx_engine.py`. This runs after ingestion, as a separate step, and does two things for each un-processed observation in the `observations` table (you decide the "unprocessed" marker – e.g., a boolean or checking if `fx_rate_used` IS NULL):

1. RECONCILIATION CHECK: verify `base_fare + taxes + mandatory_fees` is approximately equal to `total_fare_original_currency` (allow small rounding tolerance, e.g. $\pm 1\%$). If it fails this check by more than the tolerance, flag it (add a note to a `validation_flags` table, or a column – your call, propose the simplest schema addition) rather than silently accepting it. Also flag (don't delete) any `total_fare_inr` below 20% of that route's trailing 7-day median as a potential glitch fare – same "flag for review, don't auto-delete" treatment.
2. FX LOCK: for any observation where `original_currency != 'INR'`, fetch that day's RBI/FBIL reference rate (use `processing/fx_rate_fetcher.py` – write this too, it should fetch and cache the daily rate, since RBI/FBIL publish once a day) and compute `total_fare_inr = total_fare_original_currency * fx_rate`. Store `fx_rate_used` and `fx_rate_date` on the observation. For `original_currency == 'INR'` observations, just copy the value directly into `total_fare_inr` with `fx_rate_used = 1`.
3. QUALITY SCORE: compute `quality_score` as:
$$0.4 * (1 \text{ if reconciliation passed else } 0) +$$
$$0.3 * (1.0 \text{ if source=='bright_data' else } 0.8 \text{ if source=='scrappa' else } 0.8$$

if
$$\text{source=='ota_playwright' else } 0) +$$
$$0.2 * \text{recency_factor} (1.0 \text{ if collected within the last hour, decaying linearly}$$

to 0.5 at 6 hours old) +
$$0.1 * (1 \text{ if this observation has a matching independent-source observation for the same route/window/day within a reasonable price tolerance, else } 0).$$

Write this back to the `quality_score` column.

For `fx_rate_fetcher.py` specifically: fetch rates from Frankfurter's public API

(free, no API key needed), specifically the FBIL provider:

``https://api.frankfurter.dev/v2/rate/AED/INR?providers=FBIL`` for AED, and try the same pattern for SGD (``https://api.frankfurter.dev/v2/rate/SGD/INR?providers=FBIL``).

FBIL only directly publishes 7 currencies and SGD may not be among them – if the SGD call under providers=FBIL returns empty or errors, fall back to Frankfurter's default (non-FBIL) blended rate for SGD only, and log clearly which source was used for which currency. Cache each day's fetched rate locally so we don't refetch the same day's rate repeatedly (FBIL publishes once/day).

✅ **Verify:** check that international observations now have non-null `fx_rate_used`/`fx_rate_date`/`total_fare_inr`, and that the conversion math is correct (spot check one row by hand: $\text{original_amount} \times \text{fx_rate}$ should equal `total_fare_inr`).

PROMPT 8 — Imputation hierarchy

Write `processing/imputation.py`. This runs after `quality_fx_engine.py`, before the index engine. For each route + window + collection cycle where an expected observation is missing (you'll need to compare against what `routes_config.json` says should have been collected this cycle vs. what actually landed in the observations table), impute a value using this EXACT priority order, stopping at the first one that succeeds:

1. `neighbor_window` – if this route has other successful observations for different windows on the same day, use the average price-relative movement across those windows to estimate this window's value.
2. `time_carry_forward` – if step 1 has no data, reuse the same route+window's most recent successful observation (could be from an earlier collection cycle, same or previous day).
3. `cross_route` – if step 2 also fails, use the average price-relative movement of other routes in the same `domestic_international` bucket for that day.
4. `full_carry_forward` – last resort: reuse the last known absolute `total_fare_inr` for that exact route+window, unchanged.

Whichever method succeeds, insert a new observations row with `is_imputed=true`, `imputation_method` set to the method name used, and all other fields copied/derived appropriately (mark `availability_status` as whatever makes sense, e.g. `'source_error'` for the underlying cause). Never skip logging which method was used.

Also compute a `coverage_score` per route per day: the fraction of expected observations that were NOT imputed. If more than 20% of a route's observations in a day are imputed, this should be reflected as a lower `coverage_score`, which we'll surface on the dashboard as a degraded-confidence signal later.

✅ **Verify:** temporarily simulate a failure (e.g., comment out one route in a test run) and confirm the imputation script correctly fills the gap and tags it properly — check `is_imputed` and `imputation_method` columns directly in Postgres.

PROMPT 9 — Index engine (Jevons + Young, edge cases wired in)

Write `processing/index_engine.py`. This is the core statistical engine, run last

in the processing pipeline. For each collection day:

1. ROUTE-LEVEL JEVONS INDEX: for each route + window_category + advance_days combination, group all valid (non-rejected) observations sharing the same service_spec_id, compute price relatives against the base-period reference price (Day 1's median for that exact route/window/service_spec_id – store this reference separately, e.g. in a small reference_prices table or a config file, your call on the cleanest implementation), then compute the Jevons geometric mean: $(\text{product of price relatives})^{(1/n)} * 100$. Write the result into route_index (route_id, window_category, advance_days, date, price_relative, n_observations).
2. DOMESTIC/INTERNATIONAL SPLIT: compute DAPIx as the Young-type weighted sum of domestic routes' route_index values (weight = route_weight from the routes table) for a given window_category+date. Compute IAPIx the same way but for international routes only. These are two SEPARATE calculations – never mix domestic and international routes into one weighted sum.
3. OVERALL APIX: combine DAPIx and IAPIx using domestic/international weights (propose reasonable prototype weights based on relative route counts if we haven't set explicit ones – flag this assumption clearly in a comment).
4. Write results into national_index (date, window_category, domestic_apix, international_apix, overall_apix, status, coverage_score, confidence_score).
Set status='live' for the current day, 'mtd' for the current month's running aggregate, 'finalized' only once a month has fully closed (all its days have both a database entry and no pending imputation).
5. EDGE CASE – FLIGHT EVENTS: before all of the above, check the flight_events table for any operational_status other than 'completed' matching flights in today's batch, and confirm these are NEVER used to exclude or adjust the price in observations – flight_events is informational only, completely separate from this pricing calculation. Add a code comment explicitly stating this separation so it's clear to anyone reading the code later.
6. EDGE CASE – SEASONAL ROUTES: for is_seasonal=true routes, only include

```
them
in DAPIx/IAPix calculations when today's date falls within their
season_window; otherwise skip them silently (their absence shouldn't lower
coverage_score, since it's expected).
```

✅ **Verify:** query `(national_index)` after a few collection cycles and confirm you're getting sensible numbers (should hover near 100 early on, moving gradually). Confirm `(domestic_apix)` and `(international_apix)` are genuinely different numbers, not identical (a sign they got accidentally blended).

MANUAL STEP — Deploy Cron Job 2 to Render

Create a second Render Cron Job, same schedule (`(0 */4 * * *)`) but offset ~15 minutes after the first, e.g. `(15 */4 * * *)`, running a small script that calls `(quality_fx_engine.py)` → `(imputation.py)` → `(index_engine.py)` in sequence. Ask Antigravity to write this orchestrator script (`(processing/render_cron_entrypoint.py)`) if it doesn't already exist from the prompts above.

✅ **Verify:** after both cron jobs have run once, `(national_index)` has at least one row with real numbers.

PROMPT 10 — FastAPI backend + PolicyAPI

Build the `api/` folder as a FastAPI application:

1. `api/main.py` – the FastAPI app instance, includes all routers, enables automatic OpenAPI docs at `/docs` and `/redoc` (default FastAPI behavior – just don't disable it).
2. `api/routers/apix.py` – endpoints:
 - `GET /apix/daily?window={cpi_compatible|analytical}&date=YYYY-MM-DD`
 - `GET /apix/weekly?window=...`
 - `GET /apix/monthly?window=...&month=YYYY-MM`Each queries `national_index` and returns JSON.
3. `api/routers/routes.py` – `GET /routes/{route_id}/history?window=...&from=...&to=...`
 - queries `route_index` for that route's price relative history.
4. `api/routers/coverage.py` – `GET /coverage?date=...` and `GET /confidence?date=...`
 - return `coverage_score/confidence_score` from `national_index`.
5. `api/middleware/api_key_auth.py` – a FastAPI dependency that checks an `X-API-Key` header against a comma-separated list of allowed keys from the `API_KEY_ALLOWED_LIST` environment variable. Apply this dependency ONLY to the `policy_export` router (below), not the general dashboard-facing endpoints.
6. `api/routers/policy_export.py` – the NSO/RBI-facing endpoints, protected by `api_key_auth`:
 - `GET /policy/apix.csv?window=...&from=...&to=...` – full APIx time series as CSV (use Python's `csv` module or `pandas` `to_csv`, streamed as a text/csv response)
 - `GET /policy/apix.json?window=...&from=...&to=...` – same data as JSON
 - `GET /policy/routes.csv` and `/policy/routes.json` – full route-level breakdown
7. `api/models/` – Pydantic models matching each table for request/response validation.
8. `api/requirements.txt` – `fastapi`, `uvicorn`, `psycpg2-binary` (or `asyncpg`), `python-dotenv`, `pandas`.

Use `NEON_DATABASE_URL` from environment for the DB connection throughout.

✓ **Verify:** run `uvicorn api.main:app --reload` locally, open `http://localhost:8000/docs`, and manually try each endpoint through the Swagger UI. Confirm `/policy/apix.csv` correctly rejects requests without a valid API key (test with and without the header).

MANUAL STEP — Deploy FastAPI to Render

1. Create a Render **Web Service**, connect the repo, set the start command to `uvicorn api.main:app --host 0.0.0.0 --port $PORT`.
2. Add environment variables: `NEON_DATABASE_URL`, `API_KEY_ALLOWED_LIST` (pick a demo key now, e.g. a random string — this is what you'll show judges as the "institutional API key").
3. Once deployed, visit `https://your-render-url.onrender.com/docs` to confirm it's live.

✓ **Verify:** hit a real endpoint from your browser or curl against the live Render URL, confirm real data comes back.

PROMPT 11 — Dashboard scaffold + chart components

Scaffold the dashboard/ folder as a Next.js app with TailwindCSS and Framer Motion:

1. Initialize Next.js (TypeScript), install and configure Tailwind, install framer-motion, recharts (for the charts) or a charting library of your choice – propose one and explain why briefly.
2. dashboard/src/api-client.ts – a small typed client wrapping fetch calls to our deployed FastAPI backend (base URL from an env variable), with functions like getDailyApix(), getRouteHistory(routeId), getCoverage().
3. dashboard/src/components/ApixTrendChart.tsx – a line chart of national APIx over time, with a visible LIVE/MTD/FINALIZED badge per data point based on the status field from national_index.
4. dashboard/src/components/RouteRankingChart.tsx – horizontal bar chart, routes sorted by current price relative, most expensive at top.
5. dashboard/src/components/HikesDropsChart.tsx – diverging bar chart showing day-over-day % change per route, hikes to the right in red/warm color, drops to the left in green/cool color.
6. dashboard/src/components/LeadTimeCurveChart.tsx – line chart showing price relative across the T+1 through T+60 lead-time windows, either one line per route or an averaged curve (your call, propose one).
7. A main dashboard page wiring all of the above together with real data from api-client.ts, styled with Tailwind, using Framer Motion for panel entrance animations and a count-up animation on the main APIx number.

Use environment variable NEXT_PUBLIC_API_BASE_URL for the backend URL.

✅ **Verify:** run `(npm run dev)` locally, confirm the dashboard loads and shows real numbers pulled from your live Render API — not placeholder/mock data.

PROMPT 12 — Globe component

Add `dashboard/src/components/RouteGlobe.tsx` using `react-globe.gl`.

Requirements:

1. Renders a rotatable, draggable 3D globe.
2. Plots our 12+2 routes as arcs between their origin/destination airport coordinates (you'll need a small static lookup of lat/lng for DEL, BOM, BLR, CCU, HYD, MAA, AMD, DXB, SIN, GOI, SXR – look these up and hardcode them, they don't change).
3. Color each arc based on that route's current `price_relative` from our API (fetch via `api-client.ts`) – a gradient from green (below 100, cheaper than base) to red (above 100, more expensive).
4. On hover or click, show a small popup card with: route name, current price relative, direct/CPI-compatible badge, last updated timestamp.
5. Place this globe as the main visual on the dashboard's landing view.

Install `react-globe.gl` and any peer dependencies it needs.

✅ **Verify:** confirm the globe renders, rotates smoothly, and the arcs show real colors/data from your API — not static/placeholder values.

MANUAL STEP — Deploy dashboard to Vercel

1. Create a new Vercel project from the repo, root directory `dashboard/`.
2. Name the project `flywise` (or `flywise-apix` if taken).
3. Add environment variable `NEXT_PUBLIC_API_BASE_URL` pointing to your live Render API URL.
4. Deploy.

✅ **Verify:** visit your live `flywise.vercel.app` URL, confirm the globe and charts load with real data.

PROMPT 13 — Documentation

Write these judge-facing docs in docs/, based on everything we've built:

1. docs/methodology.md – explains the Jevons + Young formula, the DAPIx/IAPix split, constant-quality pricing via FSID, the base-period choice (Day 1), and the CPI-compatible (T+21/T+60) vs analytical window distinction.
2. docs/edge_cases.md – one section per edge case (diverted flights, partner substitution, imputation hierarchy, session contamination, codeshare dedup, glitch fares, flight renumbering, seasonal routes) explaining what it is and how our system handles it.
3. docs/limitations.md – honest list of what the PS's Expected Solution asks for that we've explicitly descoped or simplified for this prototype (e.g., route weights are DGCA-traffic-proxy not true expenditure weights, base period is 1 day not multiple months, only 12+2 routes not a full national basket, etc.)
4. docs/policy_api_guide.md – how NSO/RBI would consume our API: base URL, the X-API-Key header requirement, each /policy/ endpoint with a sample request and response, and a note on rate limiting.
5. docs/judge_qa_prep.md – a Q&A doc anticipating likely judge questions (why not scrape every route, how do you handle dynamic pricing, why Jevons, what happens when a source fails, how is this different from the official MoSPI CPI) with concise, honest answers based on everything we've built.

Base all of this strictly on what we actually built – don't describe features that don't exist in the code.

✅ **Verify:** read through each doc yourself and confirm every claim matches what's actually running — this is your last line of defense against overclaiming to a judge.

Both previously-open items are now resolved

- **OTA site:** Ixigo first, EaseMyTrip as fallback — MakeMyTrip and Goibibo are confirmed out due to Akamai bot protection.
- **RBI/FBIL endpoint:** Frankfurter's free API (`api.frankfurter.dev`), FBIL provider — see Prompt 7.

Nothing left to decide before you start running these prompts.